

CertiQ Dispatcher

z3 Formal Definition

CertiQ Project

June 2, 2026

Contents

Preface	3
1 Architecture Principles	3
1.1 Design Target	3
1.2 Architectural Discipline	4
1.3 Three-Layer Architecture	4
1.3.1 Geometry Layer	4
1.3.2 Proposal Layer	4
1.3.3 Certificate Layer	4
1.4 Required Invariances	5
1.5 First-Class Certificate Boundary	5
1.6 What Makes The Design Elegant	5
2 Formal Dispatch Model	5
2.1 Resources	5
2.2 Arrivals And State	6
2.3 Actions	6
2.4 Service Model	6
2.5 Objective	6
2.6 Adapter Contract	7
3 CertiQ Dispatcher Architecture	7
3.1 Canonical Equation	7
3.2 Resource Coordinates	7
3.3 Base Certified Geometry	7
3.4 Learned Assignment Proposal	8
3.5 Raw Preference For Learning	8
3.6 Candidate Mixture	8
3.7 Certificate Operator	8
3.8 Architecture Presets	9
4 Certificate Layer	9
4.1 Certificate Quantity	9
4.2 Admissible Dispatch Set	9
4.3 Tail Fallback Operator	9
4.4 Drift-Envelope Projection Operator	10
4.5 Diagnostics Contract	10
4.6 Invalid Certificate Patterns	11

5	Training And Objectives	11
5.1	Training Is Not Certification	11
5.2	Objective Hierarchy	11
5.3	Suggested Loss Components	11
5.4	Proposal Learning	12
5.5	Evaluation	12
5.6	Ablations	12
5.7	Curriculum	13

Preface

This document defines the z3 CertiQ Dispatcher architecture.

The purpose of z3 is not to patch the old model names. It is to state the architecture from first principles before any implementation rewrite happens. The core object is a certified online dispatcher for assigning arriving work to heterogeneous capacity-constrained resources.

z3 Thesis

CertiQ Dispatcher is a certified differentiable architecture for online assignment:

$$\pi^\Theta(Q, \mu, \xi) = \mathcal{C}_Q(\mathcal{A}_\Theta(Q, \mu, \xi), \mathcal{B}(Q, \mu)). \quad (1)$$

Here:

- (i) $\mathcal{B}$ is the certified base dispatch geometry;
- (ii) $\mathcal{A}_\Theta$ is the learned assignment proposal;
- (iii) $\mathcal{C}_Q$ is the state-dependent certificate operator;
- (iv) π^Θ is the final dispatch distribution.

The architecture is not defined by preview implementation variants. It is the composition of base geometry, learned proposal, and certificate operator.

Relationship to z2

The z2 material remains the current proof package for the CTMC model:

- docs/z2/formal_math/01_backbone_stability_and_constant.md
- docs/z2/formal_math/02_gate_inheritance_theorems.md

z3 reuses proven z2 facts where they apply, but it changes the architecture language. In z3, certificate mechanisms are first-class operators, not gates attached after a neural policy has already been designed.

Non-Goals

This document does not implement the rewrite. It also does not prove new stability theorems beyond the z2 proof package. New claims introduced by z3 must remain theorem obligations until separately proved.

1 Architecture Principles

1.1 Design Target

CertiQ Dispatcher is designed for one primitive:

assign one arriving unit of work to one resource, online, under heterogeneous service capacity, while preserving an auditable stability certificate.

The architecture should be judged by whether each component helps this primitive. A component is valid only if it has a clear role in at least one of:

1. representing resources without depending on arbitrary labels;

2. comparing current workload against capacity;
3. proposing a performance-improving assignment;
4. enforcing a certificate before dispatch;
5. exposing diagnostics that allow the certificate to be audited.

1.2 Architectural Discipline

The lesson to borrow from the Transformer is not the attention mechanism itself. The lesson is architectural compression: define a small number of operations that directly match the task primitive, then reuse those operations consistently.

For CertiQ Dispatcher, the primitive is not token-to-token contextualization. It is capacity-aware assignment. Therefore the central operation is:

$$\text{certified assignment} = \text{proposal} \quad \text{filtered through} \quad \text{state-dependent admissibility.}$$

Every z3 block must answer:

1. What mathematical object does this block produce?
2. What invariance or constraint does it preserve?
3. What failure mode does it prevent?
4. What diagnostic can verify that it behaved correctly?

1.3 Three-Layer Architecture

The architecture has three layers.

1.3.1 Geometry Layer

The geometry layer maps queue state and capacity into a stable dispatch energy. It defines the coordinates in which backlog is compared:

$$y_i(Q) = Q_i / \mu_i^\beta.$$

It also defines the certified base policy p^{cert} . This policy must have a theorem-facing arrival envelope:

$$A_{p^{\text{cert}}}(Q) \leq m(Q) + C.$$

1.3.2 Proposal Layer

The proposal layer learns performance corrections. It may use context ξ , attention, pooling, learned costs, compatibility features, or other domain signals.

The proposal layer is not the source of certification. It is the source of adaptivity and performance.

1.3.3 Certificate Layer

The certificate layer maps an unconstrained or partially constrained proposal into a certified final distribution.

The certificate layer is where the architecture earns the name CertiQ Dispatcher. If the final policy violates the certificate, the architecture has failed, even if training loss improves.

1.4 Required Invariances

Resource labels have no intrinsic meaning. The architecture must be permutation equivariant:

$$\pi(\sigma Q, \sigma \mu, \sigma \xi) = \sigma \pi(Q, \mu, \xi).$$

This requirement affects all blocks:

1. local feature maps must be shared across resources;
2. global summaries must be permutation invariant;
3. resource scores must be produced equivariantly;
4. the certificate operator must commute with resource permutation;
5. diagnostics must reorder consistently with the state.

1.5 First-Class Certificate Boundary

z3 forbids treating certification as a training regularizer.

The final action distribution must satisfy the certificate at inference time by construction. A drift penalty may help training, but it cannot substitute for projection, fallback, or another exact certificate operator.

1.6 What Makes The Design Elegant

An elegant CertiQ Dispatcher architecture should have:

1. one canonical forward equation;
2. one resource-set interface;
3. one diagnostics object;
4. pluggable proposal encoders;
5. pluggable certificate operators;
6. theorem claims stated only at the certificate boundary.

The implementation may expose multiple model presets, but those presets must be parameterizations of the same architecture.

2 Formal Dispatch Model

2.1 Resources

There are $N \geq 1$ resources. Resource i has service capacity $\mu_i > 0$. The capacity vector is:

$$\mu = (\mu_1, \dots, \mu_N).$$

Total service capacity is:

$$\Lambda = \sum_{i=1}^N \mu_i.$$

Each resource may carry context:

$$\xi_i \in \mathbb{R}^{d_\xi}.$$

The context may describe metadata, cost, compatibility, location, energy, hardware, or other domain information. Context does not automatically enter the certificate. It enters certification only through quantities explicitly named in a theorem.

2.2 Arrivals And State

Jobs arrive online. In the primary certifiable model, arrivals are Poisson with rate:

$$\lambda > 0.$$

The state is:

$$Q = (Q_1, \dots, Q_N) \in \mathbb{Z}_+^N,$$

where Q_i is the number of active or queued jobs at resource i .

The primary CTMC certificate assumes:

$$\lambda < \Lambda.$$

This is the subcritical load condition. It is necessary for the aggregate system but does not certify arbitrary policies.

2.3 Actions

At each arrival, the dispatcher observes:

$$(Q, \mu, \xi)$$

and outputs:

$$\pi(Q, \mu, \xi) \in \Delta_N,$$

where:

$$\Delta_N = \{p \in \mathbb{R}_+^N : \sum_i p_i = 1\}.$$

The arriving job is assigned to resource i with probability π_i .

2.4 Service Model

For the primary certifiable CTMC model, service completions are exponential. When $Q_i > 0$, resource i completes one job at rate μ_i .

The generator under policy π is:

$$(\mathcal{L}_\pi f)(Q) = \lambda \sum_i \pi_i(Q, \mu, \xi) [f(Q + e_i) - f(Q)] + \sum_i \mu_i \mathbf{1}_{\{Q_i > 0\}} [f(Q - e_i) - f(Q)].$$

The policy controls only the arrival term. Service dynamics are fixed by μ .

2.5 Objective

The default performance objective is average queueing cost:

$$J(\pi) = \limsup_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}_\pi \left[\int_0^T c(Q(t), \mu, \xi) dt \right].$$

The canonical cost is total backlog:

$$c(Q) = \sum_i Q_i.$$

Weighted costs are allowed when weights are positive and bounded:

$$c(Q, \mu, \xi) = \sum_i w_i(\mu, \xi) Q_i.$$

Certification is about stability and admissibility. It is not a proof of optimality.

2.6 Adapter Contract

A domain adapter maps a real system into:

$$\mathcal{D} = (N, \lambda, \mu, \xi, Q, c).$$

An adapter must state:

1. what counts as an arrival;
2. what counts as a resource;
3. how μ_i is measured;
4. what Q_i counts;
5. what service law is assumed;
6. what context ξ_i means;
7. what cost is optimized;
8. whether the CTMC certificate applies exactly.

If the adapter violates the CTMC assumptions, CertiQ Dispatcher may still be used empirically, but the CTMC theorem must not be claimed.

3 CertiQ Dispatcher Architecture

3.1 Canonical Equation

The z3 architecture is:

$$\pi^\Theta(Q, \mu, \xi) = \mathcal{C}_Q(\mathcal{A}_\Theta(Q, \mu, \xi), \mathcal{B}(Q, \mu)).$$

This equation defines the architecture.

$\mathcal{B}$ is the base certified geometry. $\mathcal{A}_\Theta$ is the learned assignment proposal. $\mathcal{C}_Q$ is the state-dependent certificate operator. The final policy π^Θ is the only distribution used for dispatch.

3.2 Resource Coordinates

The architecture begins by computing capacity-normalized workload:

$$y_i(Q) = Q_i / \mu_i^\beta, \quad m(Q) = \min_i y_i(Q).$$

These coordinates are not optional metadata. They are the Lyapunov geometry used by the certificate layer.

3.3 Base Certified Geometry

The base geometry produces logits:

$$u_i^{\text{cert}} = \gamma \log \mu_i - \alpha \frac{Q_i + c}{\mu_i^\beta}.$$

The base policy is:

$$p^{\text{cert}} = \text{softmax}(u^{\text{cert}}).$$

For the z2 CTMC theorem, this is the analytic backbone and satisfies:

$$A_{p^{\text{cert}}}(Q) \leq m(Q) + C.$$

In z3 language, the important object is not the old name “backbone”. The important object is a certified base distribution with an arrival-envelope constant.

3.4 Learned Assignment Proposal

The proposal layer builds an equivariant score correction from resource-local and global information.

Local features:

$$s_i = [\log(1 + Q_i), \log \mu_i, y_i, \mu_i/\Lambda, \xi_i].$$

Shared local encoding:

$$z_i^0 = f_{\text{loc}}(s_i).$$

Invariant global context:

$$g = f_{\text{glob}}(\{z_i^0\}_{i=1}^N).$$

Equivariant resource update:

$$z_i^1 = f_{\text{res}}([z_i^0, g]).$$

Proposal correction:

$$r_i^\Theta = R_{\text{max}} \tanh(h(z_i^1)).$$

Proposal policy:

$$p_{\text{prop}}^\Theta = \text{softmax}(u^{\text{cert}} + r^\Theta).$$

The proposal is anchored to the certified base logits. It is not a free black-box dispatcher.

3.5 Raw Preference For Learning

The proposal layer may also produce a raw usage preference:

$$\eta_{\text{raw}} \in [0, 1].$$

η_{raw} is not a certificate. It is only the learned desire to use the proposal.

3.6 Candidate Mixture

The candidate mixture is:

$$\pi_\eta = (1 - \eta)p^{\text{cert}} + \eta p_{\text{prop}}^\Theta.$$

The certificate operator chooses or modifies η , or more generally projects the candidate policy, so the final dispatch distribution is admissible.

3.7 Certificate Operator

The certificate operator receives:

1. current state Q ;
2. base distribution p^{cert} ;
3. proposal distribution p_{prop}^Θ ;
4. raw usage preference η_{raw} ;
5. certificate constants and diagnostics.

It returns:

1. final policy π^Θ ;
2. final gate or projection variables;
3. certificate diagnostics.

The final policy must be the distribution used by the simulator, trainer, evaluator, and deployment path.

3.8 Architecture Presets

z3 may expose presets:

1. **CertiQ Dispatcher-F**: tail fallback certificate operator;
2. **CertiQ Dispatcher-P**: drift-envelope projection operator;
3. **CertiQ Dispatcher-X**: uncertified ablation.

These are presets of one architecture, not separate architectures.

The certified presets must share the same forward interface and diagnostics contract.

4 Certificate Layer

4.1 Certificate Quantity

For any policy π , define:

$$A_\pi(Q) = \sum_i \pi_i(Q, \mu, \xi) y_i(Q).$$

The certified envelope is:

$$B(Q) = m(Q) + C.$$

The certificate condition is:

$$A_{\pi^\Theta}(Q) \leq B(Q).$$

For the current z2 CTMC theorem, $C = C_B$, the exact constant from the analytic backbone proof.

4.2 Admissible Dispatch Set

At state Q , define the admissible simplex:

$$\mathcal{P}_{\text{cert}}(Q) = \{\pi \in \Delta_N : A_\pi(Q) \leq B(Q)\}.$$

The certificate operator must return:

$$\pi^\Theta(Q, \mu, \xi) \in \mathcal{P}_{\text{cert}}(Q).$$

This makes certification an action-space constraint, not an auxiliary loss.

4.3 Tail Fallback Operator

Define:

$$S_\beta(Q) = \sum_i Q_i / \mu_i^\beta.$$

For finite R_{cert} , the fallback operator sets:

$$\eta = \eta_{\text{raw}} \mathbf{1}_{\{S_\beta(Q) \leq R_{\text{cert}}\}}.$$

Then:

$$\pi^\Theta = (1 - \eta)p^{\text{cert}} + \eta p_{\text{prop}}^\Theta.$$

When $S_\beta(Q) > R_{\text{cert}}$, the policy equals the certified base exactly.

This operator is conservative. It certifies by making the learned proposal a finite-region perturbation of the base policy.

4.4 Drift-Envelope Projection Operator

Compute:

$$A_{\text{cert}}(Q) = A_{p^{\text{cert}}}(Q), \quad A_{\text{prop}}(Q) = A_{p_{\text{prop}}^{\Theta}}(Q).$$

If $A_{\text{prop}}(Q) \leq A_{\text{cert}}(Q)$, the proposal is no worse than the base under the envelope coordinate and may be fully allowed:

$$\eta_{\text{safe}} = 1.$$

Otherwise:

$$\eta_{\text{safe}} = \left[\frac{B(Q) - A_{\text{cert}}(Q)}{A_{\text{prop}}(Q) - A_{\text{cert}}(Q)} \right]_0^1.$$

The final usage is:

$$\eta = \min(\eta_{\text{raw}}, \eta_{\text{safe}}).$$

The final policy:

$$\pi^{\Theta} = (1 - \eta)p^{\text{cert}} + \eta p_{\text{prop}}^{\Theta}$$

satisfies:

$$A_{\pi^{\Theta}}(Q) \leq B(Q).$$

This operator certifies by pointwise envelope preservation.

4.5 Diagnostics Contract

Every certified forward pass must emit:

1. A_{cert} ;
2. A_{prop} ;
3. $A_{\pi^{\Theta}}$;
4. $m(Q)$;
5. $B(Q)$;
6. $B(Q) - A_{\pi^{\Theta}}$;
7. raw proposal usage;
8. final proposal usage;
9. projection cap when applicable;
10. fallback indicator when applicable;
11. residual magnitude;
12. final policy entropy;
13. selected resource or action distribution summary.

These are architecture diagnostics. They are not optional experiment metadata.

4.6 Invalid Certificate Patterns

The following do not certify the architecture:

1. adding a drift penalty without exact enforcement;
2. using stale probabilities for projection;
3. projecting one policy and dispatching another;
4. using smooth fallback at inference without a separate proof;
5. claiming CTMC stability for adapters that violate the CTMC model;
6. allowing approximate projection without a numerical tolerance contract and proof that the resulting error preserves drift.

5 Training And Objectives

5.1 Training Is Not Certification

Training selects useful parameters. The certificate layer defines admissible actions. These roles must remain separate.

The training objective may improve cost, imitation quality, exploration, entropy, and proposal usefulness. It must not be used as the basis for a stability claim unless the final policy is also certified by construction.

5.2 Objective Hierarchy

The z3 training objective should be organized by priority:

1. maintain exact certificate enforcement in the forward path;
2. reduce average queueing cost;
3. learn useful proposal corrections over the certified base geometry;
4. preserve resource permutation equivariance;
5. keep diagnostics auditable and non-silent.

This hierarchy prevents the learned proposal from becoming a hidden replacement for the certified architecture.

5.3 Suggested Loss Components

The training loss may include:

$$\mathcal{L} = \omega_{\text{roll}}\mathcal{L}_{\text{roll}} + \omega_{\text{bc}}\mathcal{L}_{\text{bc}} + \omega_{\text{res}}\mathcal{L}_{\text{res}} + \omega_{\text{usage}}\mathcal{L}_{\text{usage}} + \omega_{\text{ent}}\mathcal{L}_{\text{ent}} + \omega_{\text{cert}}\mathcal{L}_{\text{cert}}.$$

The certificate penalty:

$$\mathcal{L}_{\text{cert}} = \mathbb{E}[(A_{\pi^\Theta}(Q) - B(Q))_+^2]$$

should be zero when exact projection is active. If it is not zero, the implementation is wrong or the numerical tolerance is incorrectly declared.

5.4 Proposal Learning

The learned proposal should be trained as a correction over the certified base, not as an independent dispatcher.

Acceptable proposal targets include:

1. rollout-improved certified policy;
2. dynamic-programming policy on tiny systems;
3. audited queueing heuristic;
4. cost-improving residual discovered by on-policy learning.

The target must be labeled. A heuristic target is not a theorem.

5.5 Evaluation

Evaluation must report both performance and certificate metrics.

Performance metrics:

1. average queue length;
2. average cost;
3. tail backlog quantiles;
4. maximum observed backlog;
5. latency proxy where applicable.

Certificate metrics:

1. violation rate;
2. minimum drift slack;
3. average drift slack;
4. projection activation rate;
5. fallback activation rate;
6. proposal usage;
7. residual magnitude.

No result should be presented as certified without certificate metrics.

5.6 Ablations

Ablations should test the architecture thesis:

1. base policy only;
2. proposal without certificate;
3. certificate with weak proposal;
4. projection versus fallback;
5. equivariant proposal versus order-dependent proposal;
6. context-free versus context-aware proposal.

The uncertified ablation must be labeled as uncertified.

5.7 Curriculum

Recommended curriculum:

1. verify base certificate quantities;
2. train proposal to imitate base or audited heuristic;
3. enable exact certificate operator during all policy rollouts;
4. optimize rollout cost;
5. audit state banks before reporting results;
6. only then expand to approximate adapters or richer domains.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.
- [2] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. Smola. Deep sets. In *NeurIPS*, 2017.
- [3] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *ICML*, 2019.
- [4] S. P. Meyn and R. L. Tweedie. *Markov Chains and Stochastic Stability*. Cambridge University Press, 2nd edition, 2009.
- [5] L. Tassiulas and A. Ephremides. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Trans. Automatic Control*, 37(12):1936–1948, 1992.
- [6] CertiQ Project. z2 proof package. `docs/z2/formal_math/01_backbone_stability_and_constant.md`, `docs/z2/formal_math/02_gate_inheritance_theorems.md`.